marola-bench: a proposal for a WeatherBench for Brazil

One scorer, Brazilian truth, and a ranking rule written before the scores

Matheus Hoffmann

2026-10-13





What this is. A proposal, for a reader outside marola. It says what the benchmark is for, what it copies, and what it adds. It reports no result: no wave forecast has been scored.

Where the design is. The design is marola's MIP-0091, accepted by the owner on 13 October 2026 with the note "needs refinement" (Hoffmann 2026b). This paper is the short public account of it. If the two disagree, the MIP is right and this paper is out of date.

The logos. The logos of marola and of Ouroboric on the first page are trademarks of their projects. They say where this work is from and which way of working it follows. They do not say that either project endorses this text.

How to read the marks. **(v)** means that the claim was checked while this paper was written, and `REFERENCES.md` says how. **(MIP §n)** means that the claim is the design's, from that section of MIP-0091; the MIP keeps its own list of what it checked and what it did not. ⚠ means not checked by anyone yet.

Hard rules

These five are the owner's rules (Hoffmann, 13 October 2026). They are not proposals, and they are not open to an agent's choice. Work on marola-bench that breaks one is wrong, even when it works.

1. **marola-bench is implemented in Apache Pekko, in Scala 3.**
2. **marola-bench is implemented on GCP (Google Cloud), and developed with Claude Code, with an SDD (spec-driven development) framework.**
3. **Its infrastructure as code is defined with Besom.**
4. **marola-bench lives in `marola-research/marola-bench`.** The first rendered paper is there.
5. **The deployment of the Pekko cluster is cheap, and it is reliable.** Both are hard requirements, and so is its infrastructure on GCP.

What each rule meets today, read on 13 October 2026. Four of the five change the accepted design, MIP-0091. By marola's own rule, a change like that is written in a MIP first. Until MIP-0091 is

amended or a new MIP is accepted, this paper states the rules and the MIP states the older design. The rules are the owner's later word, so the rules win; the list below is what the next MIP must settle.

Rule 1: Pekko, Scala 3.

- Scala 3 is already the language: marola-app's build sets Scala 3.9.0 (`build.sbt` at commit `f769245`) **(v)**.
- Pekko is new. That build uses Kyo 1.0.0-RC7 and has no Pekko dependency **(v)**, and MIP-0091 describes the forecaster on "the same toolchain (Scala 3, Kyo RC7, JDK 25)" **(MIP §4.4)**.
- Apache Pekko is a toolkit for "highly concurrent, distributed, and resilient message-driven applications using Java/Scala", under Apache-2.0 (The Apache Software Foundation, n.d.a) **(v)**.
- Not said by the rule, and not guessed here: which parts of Pekko (actors, streams, cluster), and how Pekko and the Kyo module live together.

Rule 2: GCP, Claude Code, an SDD framework.

- GCP is new, and it meets the design head-on. MIP-0091 says "Zero cloud spend" in its header, and its requirement R17 says "Zero spend by construction … Nothing provisions": fetched forecasts are free, scoring runs on GitHub Actions, computed runs use owned hardware, and the record is a lake on B2 **(MIP header, R17, §5.2)**. With rule 2, something is provisioned and something is paid. What runs on GCP, what stays where it is, and the budget are for the MIP.
- Claude Code is already how the design was written: MIP-0091's header says "Written by: Hoffmann, with Claude Code" **(MIP header)**.
- SDD is new as a rule. MIP-0091's header has a field "Spec-kit" with the value "none" **(MIP header)**. The rule does not name the framework. GitHub's Spec Kit is one, "to help you get started with SDD", under MIT (GitHub, n.d.) **(v)**; that the field's name refers to it is ⚠. Which framework is the owner's to name.

Rule 3: Besom.

- Besom is "a Pulumi SDK for Scala", by VirtusLab, under Apache-2.0 (VirtusLab, n.d.) **(v)**. Pulumi is infrastructure as code in general-purpose languages (Pulumi Corporation, n.d.) **(v)**. So the infrastructure is written in the same language as the benchmark.
- Its own page says that Scala support is "currently in Public Beta", and its latest release is v0.5.2 of 18 September 2026 **(v)**. A beta under a rule is a risk to name, not a reason to break the rule.
- MIP-0091 defines no infrastructure as code, because it provisions nothing **(MIP R17)**.

Rule 4: `marola-research/marola-bench`.

- MIP-0091 decided "zero new repositories", with two modules in marola-app, and gave marola-research the public copy, the replay results and the reports' sources **(MIP §4.4)**.
- The rule adds no repository, which agrees with the MIP. It moves the benchmark's home from marola-app to this repository, which does not. Whether the code moves, or only the research and its papers, is for the MIP. Today this directory holds the paper and nothing else.

Rule 5: cheap and reliable.

- The design already has both words, in another form: "Zero spend by construction" and a fallback that is "the normal state" **(MIP R17, §8)**. Rule 5 keeps the two aims and changes the means: a cluster that runs all the time, on a cloud that is paid.
- No price has been computed, so "cheap" has no number yet ⚠. Section 4.4 says how the design makes the cluster cheap and reliable; the number is fixed before anything is deployed.

1. Summary

Public benchmarks made weather forecast models comparable. WeatherBench is a benchmark data set for data-driven weather forecasting (Rasp et al. 2020) **(v, title only)**. WeatherBench 2 is a benchmark for the next generation of data-driven global models (Rasp et al. 2024) **(v, title only)**. Operational WeatherBench scores operational models as they run (Brightband 2026) **(v)**.

For a person on a Brazilian beach, three things are missing in them **(MIP §4.1)**:

1. **No region is Brazil.** Operational WeatherBench shows the global mean, the tropics, the two hemispheres, North America and Europe **(v)**.
2. **No score is for waves.** Its variables are of the atmosphere **(v)**. marola forecasts sea conditions (Hoffmann et al. 2026).
3. **No truth is an instrument.** Each model is scored against its own analysis **(v)**. Over an ocean with few observations, that favours agreement with the analysis, not with the sea **(MIP §4.2)**.

marola-bench is the proposal to keep the method of those benchmarks and change those three things: Brazilian regions, wave variables, and instruments on the Brazilian coast as the truth.

2. What is copied, and from whom

The method is not new and this proposal does not claim it. It is the discipline of Operational WeatherBench's methodology page **(v)**:

- **One scorer.** Every model is scored by the same code. Operational WeatherBench uses Google Research's WeatherBench-X, which is open under Apache-2.0 (Google Research, n.d.) **(v)**.
- **A truth that is stated one time.** There, each model against its own analysis, and precipitation against ERA5 (Hersbach et al. 2020) **(v)**.
- **Fixed leads and fixed regions.**
- **A baseline that is always on the chart**: a climatology **(v)**.
- **A ranking rule that is written down.**

It also copies the licence: the dashboard and its scores are CC BY 4.0, and third-party data keep their own terms **(v)**.

3. The proposal

Each item below is a requirement of MIP-0091 (its section 4.3, R1 to R20). The numbers and names are the MIP's.

3.1 Truth is an observation at a point

Truth is what an instrument measured: weather stations, wave buoys and radiosondes on and off the Brazilian coast. Each row of truth carries its instrument, its averaging, its quality flag and its licence tier. A missing observation is a row that says "missing"; it is never replaced by a model's value **(MIP R4)**.

Directional wave buoys have been operated off the southern Brazilian coast (Pereira et al. 2017) **(v, title only)**. They are few: the design counts at most about nine buoys of the national programme at the same time **(MIP §4.2)**. The format of their files, their averaging and their latency are ⚠ **(MIP, "Not checked")**.

3.2 A licence tier on every row

Some truth is under terms that restrict redistribution. Scoring uses every tier; the published export filters by tier; and the scorecard says which cells it cannot show **(MIP §4.2)**. A reader can see which scores he can reproduce and which he cannot. The status of any technical cooperation agreement is ⚠ **(MIP, "Not checked")**, and none is claimed here.

3.3 Regions are marola's own cells

South, Southeast and Northeast, and inside them the buoy, the station and the beach. The scorecard is a projection of those cells **(MIP §4.2)**. No page compares a score against instruments with a score from Operational WeatherBench: the truths differ **(MIP R19)**.

3.4 Waves are first-class

Significant wave height, peak period and direction, with buoys as truth and their own climatology floor. Wind rows and wave rows stay apart **(MIP R5)**.

3.5 Scores that the literature already defines

- A fair continuous ranked probability score with ensembles of 16 members **(MIP R2)**; the CRPS is a proper scoring rule (Gneiting and Raftery 2007) **(v, title only)**.
- For waves, a threshold-weighted CRPS above 2.5 m, because the high sea is the one that hurts **(MIP R5)**.
- Always a climatology floor, and the reward is skill relative to it. A cell with no floor publishes no reward **(MIP R6)**.
- For 24-hour precipitation, Operational WeatherBench uses SEEPS **(v)** (Rodwell et al. 2010) **(v, title only)**.

3.6 One fixed, versioned, public protocol

Leads every 6 hours to 360 hours; scorecard days 1, 3, 5, 7, 10 and 15; ranking on the 12Z cycle; windows of 7, 30 and 90 days; a minimum number of cases. A change is a new version of the protocol, never an edit **(MIP R2)**. The parameters of the decision (section 5) are committed before the first replay: this is pre-registration **(MIP R3)**.

3.7 Few instruments, said honestly

With few buoys, most cells have too few cases for months. Such a cell names no winner. It publishes the median of the admissible models and calls it model agreement. The design says: "The fallback is the normal state", and the page must not make it look better than it is **(MIP §8)**.

4. The system: the bench, the points and the actors

This section is the owner's design of 13 October 2026, in his words where they are quoted. It is later than MIP-0091 and is not in it.

4.1 What the bench is

> "The bench are the score series performed by any Forecaster, Registered by an Actor."

A **forecaster** is anything that gives a forecast for a point: a provider that publishes a model today, and marola's own forecaster later. A **score** is one forecast of one forecaster compared with one observation. A **score series** is those scores in order of time, for one forecaster, one observation point, one variable and one lead. **The bench is the set of all the score series.**

"The bench is the final goal." The points, the actors, the cluster and the cloud exist to make the series, and nothing else is the product.

Two things follow:

- **marola's forecaster is one forecaster among the others.** The bench scores it by the same rule and registers it the same way. That agrees with the design: the forecaster reads scores and writes nothing that the scorer reads **(MIP R1)**.
- **A series is a record, not a table that is computed again.** Once a score is registered it is not edited. A new protocol version starts new series **(MIP R2)**.

4.2 Observation points are added by hand

An observation point is added, changed and removed by a person, through a CRUD API (create, read, update, delete) that is specified in Apache Pekko and Scala. There is no automatic discovery of points.

By hand, because a point is not only a position. It has an instrument, an averaging, a quality flag and a licence tier **(MIP R4)**, and a person must read the terms before a point's data is scored and shown. Today the design keeps the points in a file, `sampling-points.json` **(MIP, "Checked live")**; with this rule they become a resource with an API. How the file and the API meet is for the MIP.

4.3 Actors

One actor for each observation point. The actor:

1. **monitors its point**: it knows when the next observation is due, and it gets it;
2. **asks the providers** for their forecasts for that point;
3. **in the future, asks marola's forecaster** in the same way;

4. **registers** each score in its series.

The actor is the unit of the system. A point that is added through the API becomes an actor; a point that is removed stops one. Nothing in the system is "the batch": each point keeps its own time.

This is a change of shape. In the accepted design, a cycle runs every four hours on GitHub Actions and scores everything **(MIP §2, §5.2)**. Here the work is spread over actors that live in a cluster. The scores and their rules do not change; who runs them does.

4.4 Cheap and reliable: the design principles

The Reactive Manifesto names four properties of a reactive system (Bonér et al. 2014) **(v)**: it is responsive, resilient, elastic and message driven. Each one is used here for the two hard requirements.

Property, in the manifesto's words	What it means here	Cheap	Reliable
Message driven: "asynchronous message-passing to establish a boundary between components"	an actor for each point; an actor shares no state; a provider is asked by a message with a time limit	an actor that waits uses no processor	one failed point or one failed provider does not stop the others
Resilient: "stays responsive in the face of failure"	an actor that fails is restarted by its supervisor (The Apache Software Foundation, n.d.e); an actor's state is the events it stored (The Apache Software Foundation, n.d.d), so it can start again on another node	a node may be lost, so capacity that can be taken away is usable	no score is lost with a node
Elastic: "stays responsive under varying workload"	an actor with nothing to do is stopped and started again when needed. Pekko calls this passivation: "you may stop entities that are not used to reduce memory consumption" (The Apache Software Foundation, n.d.c) **(v)**	a point acts a few times a day and is idle the rest; memory follows the work, not the number of points	adding points does not need a larger design

Property, in the manifesto's words	What it means here	Cheap	Reliable
Responsive: "responds in a timely manner if at all possible"	back-pressure (The Apache Software Foundation, n.d.b): a slow provider slows its own stream and fills no queue	no capacity is kept for a peak	an answer that does not come in time is a row that says "missing", never a guess **(MIP R4)**

Six rules follow from them. They are proposals for the next MIP, not facts.

1. **Reliability is what makes it cheap.** Cheap capacity on a cloud is the kind that can be taken away. On GCP, "Spot VMs have significant discounts, but Compute Engine might preemptively stop or delete (preempt) Spot VMs to reclaim the capacity at any time" (Google Cloud, n.d.) **(v)**. A cluster that survives the loss of a node can use it. So the two hard requirements are one piece of work: first survive the loss of any node, then buy the cheap nodes.
2. **State is events.** An actor stores what happened (an observation came, a forecast came, a score was registered) and builds its state from that. A restart loses nothing.
3. **Every write can be repeated.** After a restart an actor may do its last step again. A score registered two times is one score.
4. **The smallest cluster that survives one loss.** Its size is a value of the protocol, like the budget **(MIP R3, R17)**, and it is written before the deployment. A cluster needs a rule for the case in which its nodes cannot see each other; Pekko has one (The Apache Software Foundation, n.d.f).
5. **The budget has a switch.** The design already has a kill switch that returns every cell to the fallback **(MIP R10)**. The same idea for cost: above the budget, the cluster does less (fewer providers, longer intervals) before it costs more, and it says so on the bench.
6. **The infrastructure is code, in the same language.** It is defined with Besom (hard rule 3), so the cluster's size and its budget are reviewed in a pull request like any other value.

What this section does not decide: which GCP products, where the events are stored, how many nodes, and the budget in money. None has been measured or priced ⚠.

5. From a scoreboard to one forecast

A scoreboard ranks. A swimmer needs one number for one beach. So the design adds a forecaster on top of the scores, and keeps it apart from them: the forecaster reads score cells and writes nothing that the scorer reads **(MIP R1)**.

The candidates (the design calls them arms) are of two kinds, and each kind has its own algorithm **(MIP R8)**:

- **Fetched arms** are forecasts that others publish. All of them are scored every cycle, so the loss of each is always seen. They are experts with full information, and the rule is Hedge.
- **Computed arms** are wave runs that cost compute, such as runs of the WW3 GPU port (Hoffmann 2026d). Only the runs that were made are seen. They are a bandit with a budget (Li et al. 2010) **(v, title only)**, and the rule is Thompson sampling with a knapsack.

The design is exact on one point: never a bandit over arms whose loss is observed in all cases **(MIP R8)**. The owner's first note called the whole thing a contextual bandit (section 10); the accepted design is narrower, and this paper follows the design.

The context of a decision is the region, the lead time, the variable and the state of ENSO, and every feature of it is dated before the forecast was issued **(MIP R9)**.

6. Conservative, and safe outside the forecaster

- **A model is admitted for correctness, not for skill.** A wave run enters only with its regression tests passed and recorded; for the GPU port that means bit for bit **(MIP R15)**.
- **The quoted model changes slowly.** A new model replaces the incumbent only when the 95% interval of a paired block bootstrap is above zero, on two consecutive weekly runs **(MIP R10)**.
- **No live exploration.** A swimmer sees only a model that passed its gate. Exploration changes which computed runs are made, never what is shown **(MIP R11)**.
- **Safety reads nothing from the forecaster.** The swimming score and the warning for a dangerous sea do not depend on it **(MIP R12)**.
- **Disagreement is shown** beside the winner **(MIP R18)**.
- **Zero spend by construction**, in the accepted design: fetched forecasts are free, scoring runs on GitHub Actions, and computed runs use owned hardware under a budget that is a value of the protocol **(MIP R17)**. Hard rule 2 puts the benchmark on GCP, so this item changes with the next MIP; a budget that is a value of the protocol is the part to keep.

7. Why this is open

Closed Source: The God that Failed argues that a result nobody can check is a result nobody should trust (Hoffmann 2026a). A benchmark is that argument at a small size: a scoreboard with a hidden scorer asks for trust. So the design has a public chain **(MIP R20)**:

- the scorecards are CC BY 4.0, with each source's terms kept;
- each version of the protocol gets a dated data set on Zenodo, with a permanent identifier, as the software citation principles ask (Smith et al. 2016) **(v)**;
- the reports and their sources are in this repository;
- the toolchain is pinned (Hoffmann 2026c), so a score can be made again.

8. The order

The design's stages **(MIP §5.6)**, in order. No date is given, because the design gives none.

1. Wave truth goes into the evaluation that already runs for wind.
2. The forecaster is written and replayed on the wind record.
3. Computed wave runs enter, after their correctness gate.
4. The product shows the winner, after the gate passed.
5. The public copy, the reports and the Zenodo data sets come to this repository, after the owner approves the publication.

These are the stages of the accepted design. The hard rules and section 4 change who runs the

work (actors in a cluster on GCP, not a cycle on GitHub Actions), so the next MIP may change the stages too. This paper does not write new ones.

This paper is before stage 5. It publishes no score and no part of the private policy note that the design cites.

9. What is not known

From the design's own lists **(MIP §8, §11, "Not checked")**:

- the format, averaging and latency of the buoy files, and the terms for each station;
- the status of any cooperation agreement, and what each partner can provide;
- which wave models are served for the Brazilian coast by the public source the design names;
- whether a skill learned in an El Niño season stays valid in a neutral year;
- which buoy is the first wave truth. The design's default is one off Santa Catarina with a continuous record.

From this paper:

- **What the five hard rules leave open**, for the next MIP: which parts of Pekko, and how it lives with the Kyo module; what runs on GCP, and its budget, against the design's zero spend; which SDD framework; and whether the code moves to this repository or only the research does.
- **What section 4 leaves open**: the GCP products, the store of the events, the number of nodes, the budget in money, and how the actors and the four-hour cycle of today meet.
- **The name.** The design calls the benchmark "marola-eval" **(MIP §11)**. The owner asked for this paper under the name "marola-bench". Which one the public sees is his decision.
- **The licence of this text.** This repository is MIT. The design wants CC BY 4.0 for what it publishes **(MIP R20)**. That is the owner's decision too.
- The claims marked "title only": the record of the paper was read, and its text was not. No claim here depends on a detail of one.

10. Where the thesis comes from

- MIP-0091, "The Marola forecaster's architecture", in `marola-dev/marola` at commit `cf7bd4f` (Hoffmann 2026b). Author: Hoffmann. Status: Accepted, 13 October 2026.
- Five notes of the owner in `h0ffmann/closed-source` at commit `bc1caaf` (https://github.com/h0ffmann/closed-source/tree/bc1caaf/notes), filed on 12 and 13 October 2026: the method to copy and its gaps (`202610122356`), the forecaster as a contextual bandit (`202610122357`), Brazilian truth (`202610122358`), exploit in operations and explore in simulation (`202610122359`), and read, evaluate, adopt for forecast models (`202610130000`).

Left out on purpose: the private policy note that MIP-0091 cites, a future version of WAVEWATCH III, and the names of the institutions behind the agreements. None has a public source.

References

How each entry was checked is in `REFERENCES.md`; the list's source is `refs.bib`.

Bonér, Jonas, Dave Farley, Roland Kuhn, and Martin Thompson. 2014. "The Reactive Manifesto." https://www.reactivemanifesto.org/.

Brightband. 2026. "Operational WeatherBench: Methodology." https://owb.brightband.com/methodology.

GitHub. n.d. "Spec Kit." https://github.com/github/spec-kit.

Gneiting, Tilmann, and Adrian E. Raftery. 2007. "Strictly Proper Scoring Rules, Prediction, and Estimation." *Journal of the American Statistical Association* 102 (477): 359–78. https://doi.org/10.1198/016214506000001437.

Google Cloud. n.d. "Compute Engine Documentation: Spot VMs." https://cloud.google.com/compute/docs/instances/spot.

Google Research. n.d. "WeatherBench-X: A Modular Framework for Evaluating Weather Forecasts." https://github.com/google-research/weatherbenchX.

Hersbach, Hans, Bill Bell, Paul Berrisford, Shoji Hirahara, et al. 2020. "The ERA5 Global Reanalysis." *Quarterly Journal of the Royal Meteorological Society* 146 (730): 1999–2049. https://doi.org/10.1002/qj.3803.

Hoffmann, Matheus. 2026a. *Closed Source: The God That Failed*. Zenodo. https://doi.org/10.5281/zenodo.23333656.

———. 2026b. "MIP-0091: The Marola Forecaster's Architecture: The Evaluation Goes First, No New Repo, and the Design Requirements from Read, Evaluate, Adopt on Operational WeatherBench." https://github.com/marola-dev/marola/blob/cf7bd4f/docs/MIPs/MIP-0091-marola-forecaster-architecture.md.

———. 2026c. "Picasso: Self-Contained Nix Flakes for Scientific and Agentic Toolchains, and the NixOS Workstation They Run On." Zenodo. https://doi.org/10.5281/zenodo.23311208.

———. 2026d. "WW3 GPU Lab: Agentic Coding and Agentic Research on a Bit-for-Bit GPU Port of WAVEWATCH III." Zenodo. https://doi.org/10.5281/zenodo.23221351.

Hoffmann, Matheus, Bruno Valério, Rob Kler Soares da Silva Junior, Elisa Oliveira, Leonardo Ramos Almeida, and Pablo Ribeiro. 2026. "Marola: An Open, Non-Profit Platform for Sea Conditions and Bathing-Water Quality at Brazilian Beaches, Built on Public Data." Zenodo. https://doi.org/10.5281/zenodo.23224155.

Li, Lihong, Wei Chu, John Langford, and Robert E. Schapire. 2010. "A Contextual-Bandit Approach to Personalized News Article Recommendation." In *Proceedings of the 19th International Conference on World Wide Web*, 661–70. https://doi.org/10.1145/1772690.1772758.

Pereira, Henrique P. P., Nelson Violante-Carvalho, Izabel C. M. Nogueira, Alexander Babanin, et al. 2017. "Wave Observations from an Array of Directional Buoys over the Southern Brazilian Coast." *Ocean Dynamics* 67 (12): 1577–91. https://doi.org/10.1007/s10236-017-1113-9.

Pulumi Corporation. n.d. "Pulumi: Infrastructure as Code in Any Programming Language." https://github.com/pulumi/pulumi.

Rasp, Stephan, Peter D. Dueben, Sebastian Scher, Jonathan A. Weyn, Soukayna Mouatadid, and Nils Thuerey. 2020. "WeatherBench: A Benchmark Data Set for Data-Driven Weather Forecasting." *Journal of Advances in Modeling Earth Systems* 12 (11): e2020MS002203. https://doi.org/10.1029/2020MS002203.

Rasp, Stephan, Stephan Hoyer, Alexander Merose, Ian Langmore, et al. 2024. "WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models." *Journal of Advances in Modeling Earth Systems* 16 (6): e2023MS004019. https://doi.org/10.1029/2023MS004019.

Rodwell, Mark J., David S. Richardson, Tim D. Hewson, and Thomas Haiden. 2010. "A New Equitable Score Suitable for Verifying Precipitation in Numerical Weather Prediction." *Quarterly*

Journal of the Royal Meteorological Society 136 (650): 1344–63. https://doi.org/10.1002/qj.656.

Smith, Arfon M., Daniel S. Katz, Kyle E. Niemeyer, and FORCE11 Software Citation Working Group. 2016. "Software Citation Principles." *PeerJ Computer Science* 2: e86. https://doi.org/10.7717/peerj-cs.86.

The Apache Software Foundation. n.d.a. "Apache Pekko." https://github.com/apache/pekko.

———. n.d.b. "Apache Pekko Documentation: Basics and Working with Flows." https://pekko.apache.org/docs/pekko/current/stream/stream-flows-and-basics.html.

———. n.d.c. "Apache Pekko Documentation: Cluster Sharding." https://pekko.apache.org/docs/pekko/current/typed/cluster-sharding.html.

———. n.d.d. "Apache Pekko Documentation: Event Sourcing." https://pekko.apache.org/docs/pekko/current/typed/persistence.html.

———. n.d.e. "Apache Pekko Documentation: Fault Tolerance." https://pekko.apache.org/docs/pekko/current/typed/fault-tolerance.html.

———. n.d.f. "Apache Pekko Documentation: Split Brain Resolver." https://pekko.apache.org/docs/pekko/current/split-brain-resolver.html.

VirtusLab. n.d. "Besom: A Pulumi SDK for Scala." https://github.com/VirtusLab/besom.